

Database Systems and Laboratory

SQL - Views

Nicola Saccomanno

`nicola.saccomanno@uniud.it`

Bachelor degrees in Internet of things, Big Data, Machine Learning, & Computer Science
DMIF, University of Udine

A.Y. 2025/2026

Introduction

Outer level (or views): provide users with different representations of the same data.

Distinction between base relations/tables and **derived tables**.

Two types of derived tables:

- ▶ Virtual relations (or views)
 - ▶ Relations defined by means of queries.
 - ▶ Have a schema, but no instance → each time we use the view, it is recomputed, leading to always up-to-date information
 - ▶ Usable in queries like base tables → enables nested queries in the FROM clause
- ▶ Materialized views, derived tables whose content (instance) is actually stored in the database
 - ▶ advantage: view usage becomes extremely lightweight
 - ▶ issue: maintaining alignment with base tables

A Simple Example

Given a database containing the following relations:

$R(A, B, C)$, $S(C, D, E)$ and $T(F, G)$.

Let V be the view defined by the query

$$V = \sigma_{A>D}(R \bowtie S)$$

.

The query $V \bowtie_{B=F} T$ is executed by substituting the view V with its definition:

$$(\sigma_{A>D}(R \bowtie S)) \bowtie_{B=F} T$$

.

Use of Views: Advantages

- ▶ Select only the components of interest:

$$\Pi_{Fname,Lname,Dname}(EMPLOYEE \bowtie_{Dno=Dnumber} DEPARTMENT)$$

- ▶ Express very **complex expressions compactly**, with repeated subexpressions.
- ▶ **Formulate otherwise unexpressible queries.**
- ▶ Introduce **privacy protection mechanisms** (access authorizations with respect to views).
- ▶ **Enable legacy support:** in the presence of changes to the schema after its first implementation (schema restructuring), define views that correspond to relations no longer present, but derivable from new relations.

Views in SQL - 1

Views are defined in SQL as follows.

```
Syntax: CREATE VIEW ViewName [(AttributeList)] AS  
        SELECT-SQL  
        [WITH [LOCAL | CASCADE] CHECK OPTION]
```

The query must return a set of attributes compatible with the attributes declared in the view's schema.

Example: Define a view EMPLOYEES5 containing only the employees of department 5 earning more than 10,000 euros.

```
CREATE VIEW EMPLOYEES5(ID5, NAME5, SURNAME5, SALARY5) AS  
    SELECT SSN, Fname, Lname, Salary  
    FROM   EMPLOYEE  
    WHERE  Dno = 5 AND Salary > 10000
```

Views in SQL - 2

From the view EMPLOYEES5, a second view can be constructed containing only the employees of department 5 with a salary between 10,000 and 25,000 euros (lower-income employees).

```
CREATE VIEW EMPLOYEES5POOR AS
  SELECT *
  FROM   EMPLOYEES5
  WHERE  SALARY5 < 25000
  WITH CHECK OPTION
```

The WITH CHECK OPTION statement, possibly qualified with LOCAL or CASCADED (CASCADED is the default), concerns the ability to modify a view (see later).

SQL Queries and Views - 1

Views can be used to draft queries that are otherwise inexpressible in SQL, thereby increasing the expressive power of the language.

- ▶ E.g., use of multiple **aggregate functions in cascade** (or non-elementary use of the union operator).

Example: Determine the department(s) characterized by the maximum total salaries paid to its members.

```
CREATE VIEW DEPTTOTALSAL(DEPT, TOTALSALARY) AS
  SELECT    Dno, SUM(Salary)
  FROM      EMPLOYEE
  GROUP BY Dno
```

```
SELECT DEPT
FROM   DEPTTOTALSAL
WHERE TOTALSALARY >= ALL (  SELECT  TOTALSALARY
                             FROM    DEPTTOTALSAL)
```

SQL Queries and Views - 2

Alternative solution:

```
SELECT    Dno
FROM      EMPLOYEE
GROUP BY  Dno
HAVING    SUM(Salary) >= ALL (      SELECT    SUM(Salary)
                                     FROM      EMPLOYEE
                                     GROUP BY  Dno)
```

This second solution is not supported by all systems (some systems impose the restriction that the condition associated with the HAVING clause does not contain nested queries).

SQL Queries and Views - 3

Example: Considering the employees with at least one dependent, determine the average and maximum number of dependents.

```
CREATE VIEW DEPENDENTEMP(EMP, DEPENDENT) AS
    SELECT    ESSN, COUNT(*)
    FROM      DEPENDENT
    GROUP BY  ESSN

SELECT    AVG(DEPENDENT), MAX(DEPENDENT)
FROM      DEPENDENTEMP
```

Incorrect alternative solution:

```
SELECT    AVG(COUNT(*)), MAX(COUNT(*))
FROM      DEPENDENT
GROUP BY  ESSN
```

In general, the cascading evaluation of two different operators occurs at different aggregation levels, but only one occurrence of the GROUP BY clause is allowed per query.

Recursive Views in SQL-3

SQL-2 does not allow recursive dependencies of any type between views.

SQL-3 adopts the method of defining recursive views from the Datalog language.

Example: Find the direct or indirect supervisors of Giovanni Rossi.

```
CREATE RECURSIVE VIEW HASSUPERIOR(EMPLOYEE, SUPERIOR) AS
  (( SELECT SSN, Super_SSN
    FROM   EMPLOYEE
    WHERE  Super_SSN IS NOT NULL)
  UNION
  ( SELECT E2.EMPLOYEE, E1.Super_SSN
    FROM   EMPLOYEE E1, HASSUPERIOR E2
    WHERE  E2.SUPERIOR = E1.SSN AND E1.Super_SSN IS NOT NULL))

SELECT   SSN, SUPERIOR
FROM     (EMPLOYEE JOIN HASSUPERIOR on
          EMPLOYEE.SSN = HASSUPERIOR.EMPLOYEE)
WHERE    Fname = 'GIOVANNI' AND Lname = 'ROSSI'
```

Recursive Views in SQL-3: step-by-step resolution I

The HASSUPERIOR recursive view contains the employees (identified by social security number) and their direct or indirect supervisors. It should be noted that there can be multiple (indirect) supervisors for each employee, so the social security number alone does not identify the tuples of the HASSUPERIOR view. In general, the concept of a key is not defined for views. The recursive definition of HASSUPERIOR specifies a base case (the direct supervisors) and a recursive case (the indirect supervisors) where the HASSUPERIOR view is reused. Finally, the target query is simply resolved by selecting the supervisors of the employee with a given name (e.g., 'GIOVANNI ROSSI').

It is instructive to analyze the method of computing a recursive view. The algorithm is similar to that which solves the reachability problem on a graph: starting from an initial set of nodes, find the set of nodes reachable from these through an arbitrary path.

1. As a first step, apply the base definition and obtain a first result X (the employees and their direct supervisors in our example).
2. At this point, apply the recursive definition where the HASSUPERIOR table contains only the tuples in X . This way, a second result Y is obtained (the employees and the supervisors of their supervisors).

Recursive Views in SQL-3: step-by-step resolution II

3. If Y contains some tuples not belonging in X , then the recursive definition is applied, with the HASSUPERIOR table containing only the tuples in $Y - X$ (i.e., the new tuples), obtaining a new result Z (the employees and the supervisors of the supervisors of their supervisors).
4. If Z contains new tuples not belonging in $X \cup Y$, then proceed similarly until a result is obtained that does not add any rows compared to what has been calculated up to that point.
5. The result of the recursive evaluation is the union set of all the partial results obtained throughout the iterative process (X united with Y united with Z , and so on).

For example, if A has B as a supervisor, B has C as a supervisor, C has D as a supervisor, and D has no supervisor (that is, his supervisor is NULL), then the first iteration produces the set of pairs (employee, supervisor) equal to $X = (A,B), (B,C), (C,D), (D, \text{NULL})$. The second iteration produces the set $Y = (A,C), (B,D)$. The third iteration produces $Z = (A,D)$. The subsequent iteration produces nothing and thus the final result is the union of X , Y , and Z , i.e., $(A,B), (B,C), (C,D), (A,C), (B,D), (A,D)$

Views and Update Operations I

Some views allow update operations that are transformed into corresponding operations on the base tables from which such views depend.

It is not always possible to uniquely trace modifications on views back to changes in base tables (trivial case: views defined using aggregate functions).

There are many intricate scenarios...

```
CREATE VIEW WORKS_ON_PROJECT AS
```

```
    SELECT Fname, Lname, Pname, Hours  
    FROM   EMPLOYEE, PROJECT, WORKS_ON  
    WHERE  SSN = ESSN AND Pnumber = Pno
```

```
UPDATE WORKS_ON_PROJECT
```

```
SET      Pname = 'PROJECTB'
```

```
WHERE  Lname = 'ROSSI' AND Fname = 'MARIO' AND Pname = 'PROJECTA'
```

Views and Update Operations II

Issues with the previous example:

- ▶ Let's assume that the DB states that Mario Rossi works at (15, PROJECTA). The result of the update on the view could mean (15, PROJECTA) $\rightarrow$ (15, PROJECTB)
- ▶ Let's assume the DB states that Mario Rossi works at (15, PROJECTA) and it also contains (18, PROJECTB). The result of the update on the view could mean that Mario Rossi now works on (18, PROJECTB)
- ▶ Impossible to determine semantic coherence!

When Can a View Be Modified?

- ▶ **Standard SQL:** A view is updateable only when a single tuple of each base table corresponds to a tuple of the view (this, for example, implies that the view cannot involve aggregate functions)
- ▶ **Commercial systems** are more restrictive: a view is updateable only if it is defined on a single table (some systems also require that the view's attributes include the primary key of the table and that all attributes of the view are associated with the NOT NULL constraint but not a default value).
 - ▶ The CHECK OPTION clause can be used only in the context of such views: it specifies that only updates of the tuples of the view are allowed and imposes that after their execution, such tuples continue to **belong to the view** (a tuple can be removed from the view if a value is assigned to one of the view's attributes that makes the condition of the WHERE clause false).

Example: If we assume the view EMPLOYEES5, it is not possible to operate a salary reduction for an employee of Department 5 from a value above 10,000 euros to a value less than or equal to 10,000 euros.

LOCAL or CASCADE Option

In the case a view is defined in terms of other views:

- ▶ **LOCAL option:** The check on maintaining the tuples' membership in the view is done only at the last level (the modification must not cause a violation of the condition defining the outermost view);
- ▶ **CASCADED option** (default option): The check on maintaining the tuples' membership in the view is done at all levels (it is checked that the tuples being modified do not disappear from the view as a result of violating any of the conditions of the involved views).

Examples: An assignment of the value 8,000 euros to one of the tuples of the view *EMPLOYEES5POOR* is accepted with the LOCAL option, but is rejected with the CASCADED option (default). A modification that assigns a value of 40,000 euros to a tuple of the view is rejected with both options.

A partially similar approach to views: CTE and WITH

Common Table Expression (CTE) - named query introduced by the WITH clause.

- ▶ It defines a **temporary derived table** that can be used only by the immediately following query.
- ▶ Conceptually, it is similar to a view, but it is **not stored in the schema**.
- ▶ It is useful to decompose complex queries into simpler steps.
- ▶ Recursive CTEs are defined with WITH RECURSIVE.

Example: the query used before can be written without creating a persistent view.

```
WITH DEPTTOTALSAL(DEPT, TOTALSALARY) AS (  
    SELECT    Dno, SUM(Salary)  
    FROM      EMPLOYEE  
    GROUP BY Dno )  
SELECT  DEPT  
FROM    DEPTTOTALSAL  
WHERE   TOTALSALARY >= ALL ( SELECT TOTALSALARY  
                             FROM    DEPTTOTALSAL)
```

References

Material for these slides is largely based on:

- ▶ Prof. Angelo Montanari's slides about views

<https://users.dimi.uniud.it/~angelo.montanari/Viste.pdf>